

Chapter 3.RecordCache Design Spec v2

Background

在使用 zipfian 分布(倾斜参数 $a = 0.99$)模拟数据库 larger-than-memory 的 OLTP 负载任务时，对应有如下的参数：

访问倾斜程度设置 a (item_size = 16 KB) Page	Top 50% 查询请求访问的数据量 (GB)	Top 80% 查询请求访问的数据量 (GB)	Top 90% 查询请求访问的数据量 (GB)	Top 95% 查询请求访问的数据量 (GB)	Top 99% 查询请求访问的数据量 (GB)
$a = 0.99$	0.15	33.99	194.97	461.69	915.28

将内存从 2GB 扩展到 195 GB 时，能够 cache 住近 90% 的查询请求，但是如果希望 cache 住 90% 到 99% 的查询请求，需要把内存从 195 GB 扩展到 915 GB。这额外的 720 GB 能够缓存住查询的长尾部分，但是 DRAM 资源价格较为昂贵。

扩展 720 GB 内存，将缓存命中率从 90% 提升到 99% 时，系统的吞吐量加速比将达到 2.74(按照缓存命中的查询时间是 0.1 us, 缓存未命中读磁盘的时间是 5 us)，但具有较高的成本。

在引入 CXL 作为扩展内存之后，Tier 1 的 DRAM 访问时间约为 0.1us, Tier 2 的访问时间约为 0.2 us, Tier 3 的访问时间约为 5 us。把 720 GB 内存全部替换成 CXL-Attached DRAM 资源后，尽管增加了访问时间的延迟，依然能够把系统吞吐量加速比提升到 2.63。

=> CXL 扩展内存的好处，低成本缓存住倾斜工作负载情况下长尾的部分，在 Tier2 (CXL) 和 Tier 3(SSD) 访问延迟存在数量级差距的情况下，依然能够提升 2.63 倍的系统加速比，对 larger-than-memory 的工作负载展现出 in-memory 的查询性能，对于 Tier1(DRAM) 部分的容量显著不足时，能够显著提升系统的命中率。(Background)

我们的工作建立在三级存储层次结构上的缓存系统设计，Tier 1(DRAM) -> Tier 2(CXL) -> Tier 3 (SSD)。

在三级存储系统结构的层次上，访问延迟最低，存储最热数据，最显著提升系统整体吞吐量的 DRAM 内存，在原先二级存储层次结构 DRAM(Tier1) -> SSD(Tier2) 中存在读放大的问题，根本原因是 SSD 的读写单元按照 block 组织，粒度为 16 KB。对于 Cold Page 中存在少量 Hot Record 的情况，二级存储层次结构的做法是在 DRAM 中维护 Page 粒度的 Buffer Pool, 这带来内存空间的浪费。

同时，由于缓存驱逐策略是以 page 粒度统计，因此很难区分出在 $\text{page_visit_count} = N$ 的情况下，究竟是对当前 page 上的同一条 record 进行了 N 次访问(Cold Page, Hot Record)还是对当前 page 上接近 N 条不同的 record 进行了访问。(Hot Page)。

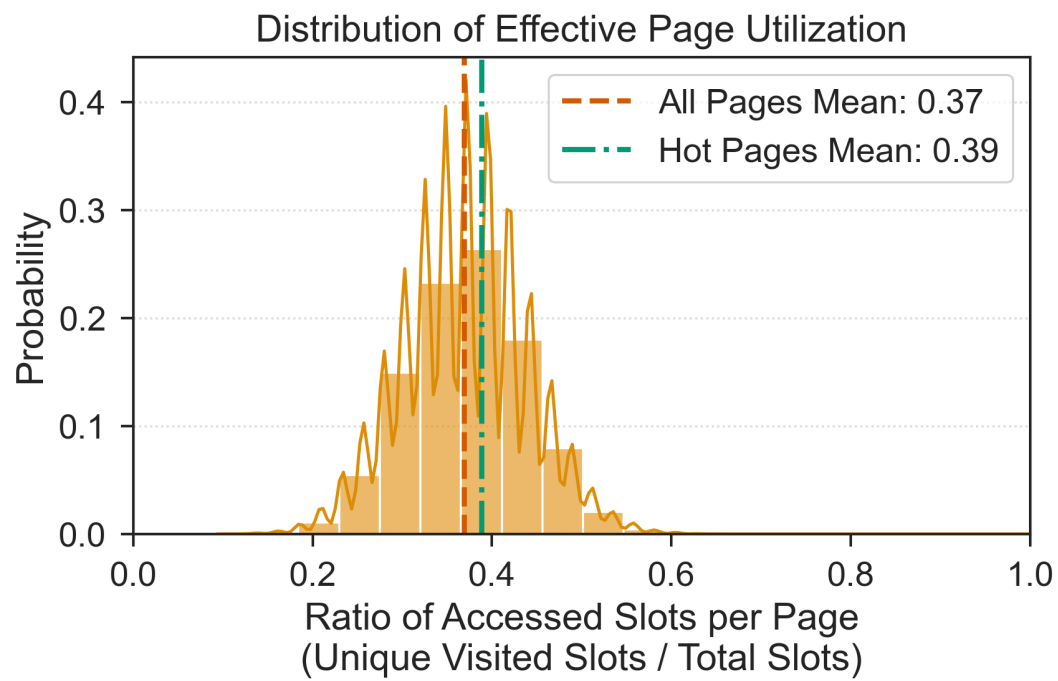
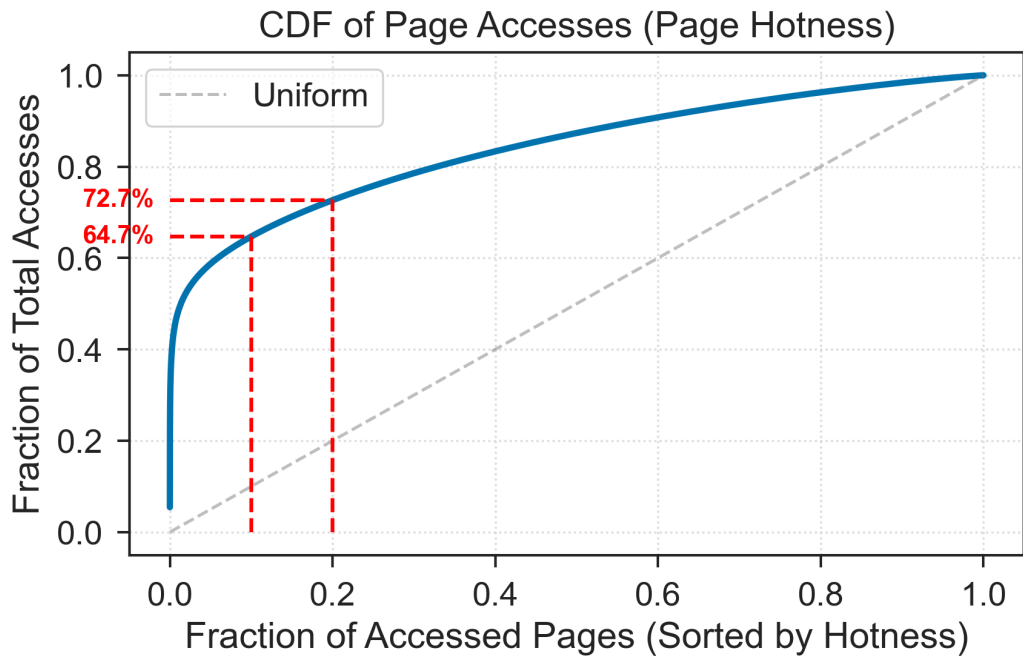
在引入可字节粒度寻址的 CXL 存储层次之后，我们提出在 DRAM 中维护 RecordCache, 在 CXL 中维护 Page-Sized Buffer Pool。较为根本的区分 page 和 record 的命中访问统计，进一步高效利用 Tier 1 的 DRAM 资源。(在使用 Page 粒度进行访问统计 vs Record 粒度进行访问统计，下图展示出使用 RecordCache 缓存住 90% 的请求访问时，相比 Page 粒度能够减少 57.45 的 Tier 1 DRAM 资源使用)。

访问倾斜程度设置	Top 50% 查询请求访问的数据量(GB)	Top 80% 查询请求访问的数据量(GB)	Top 90% 查询请求访问的数据量(GB)	Top 95% 查询请求访问的数据量(GB)	Top 99% 查询请求访问的数据量(GB)
$a = 0.99$ (16 KB Page)	0.15	33.99	194.97	461.69	915.28
$a = 0.99$ (Record 242 B)	0.02	16.68	137.52	388.40	884.41

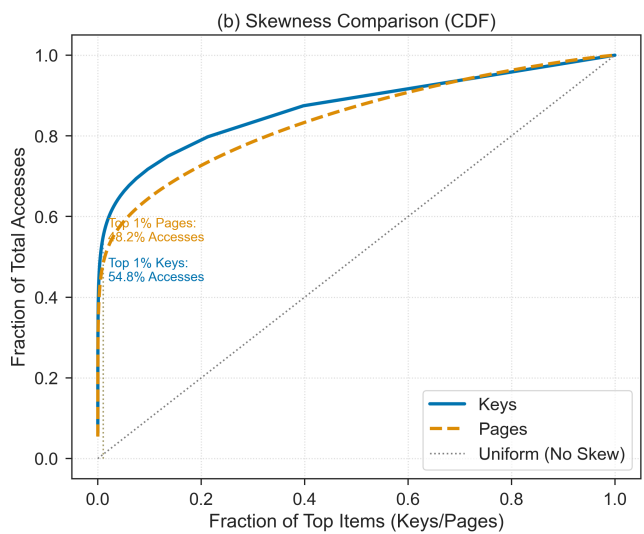
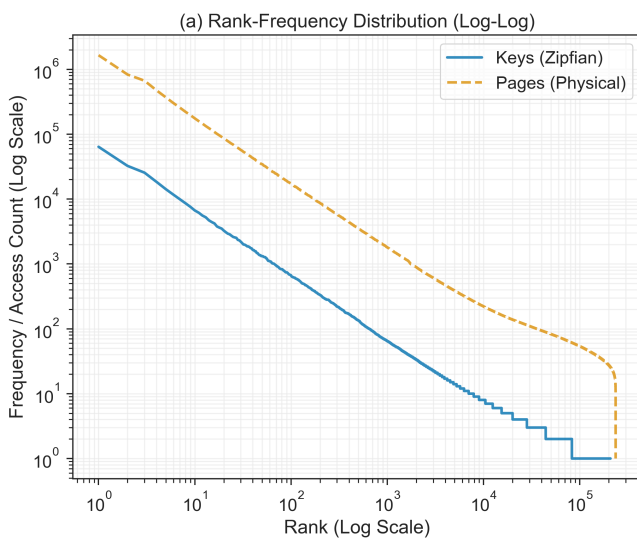
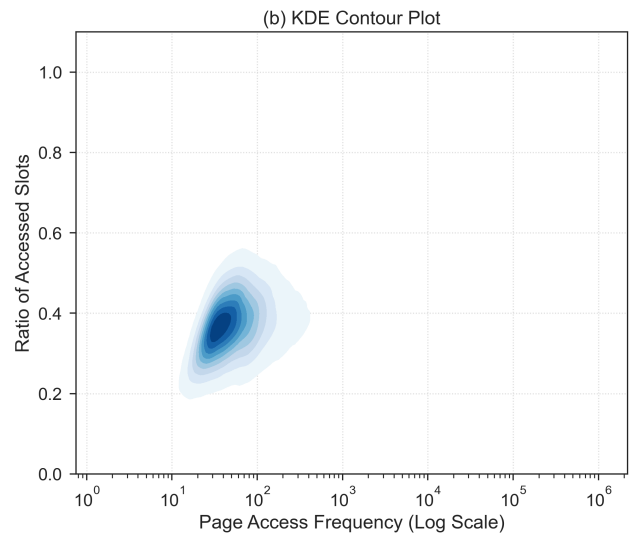
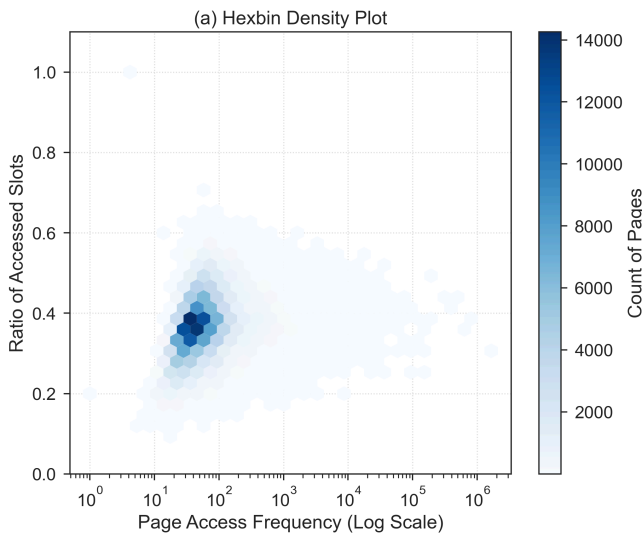
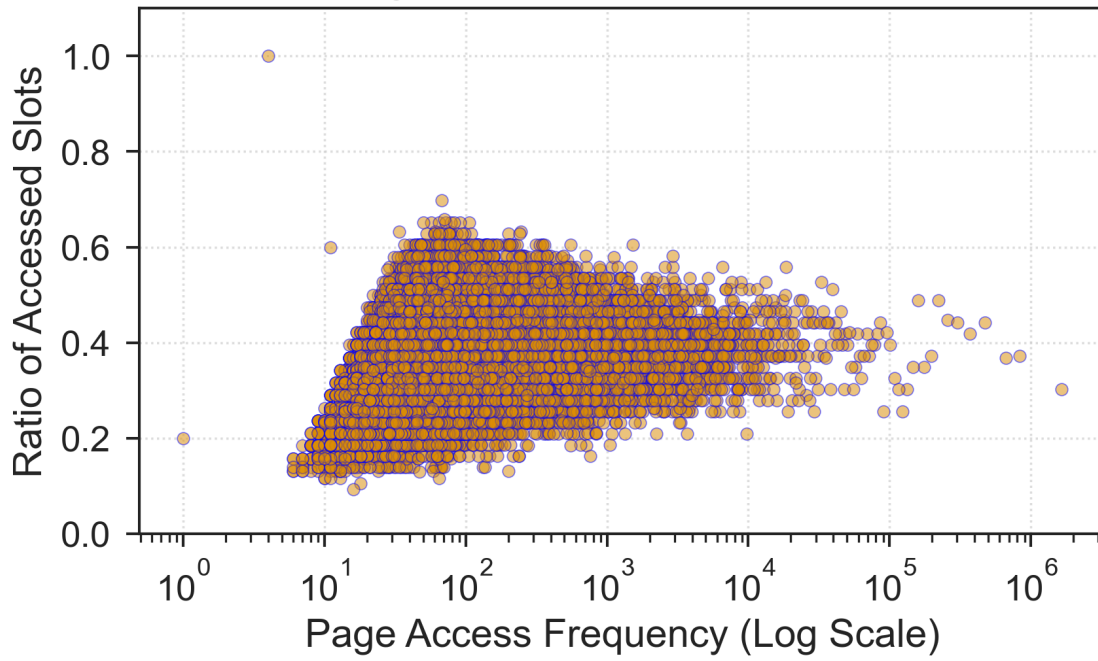
当前设计的挑战:

1. 适配 RecordCache 的高效 Page 识别机制，对 (Cold Page, Hot Record) 和 (Hot Page) 的 page 粒度访问统计的负载区分，在访问延迟存在差异的情况下(100 ns vs 200 ns), 动态调整 RecordCache vs CXL Buffer Pool 的内存大小。
2. 混合粒度缓存管理的一致性维护问题，数据源的两个版本分别在 RecordCache 的 record 和 CXL Buffer Pool 的 Page 上。RecordCache 对 point-select 等随机数据访问有明显的加速效果，但 scan 访问依然需要在有序索引的情况(B+Tree)下进行, 在数据源存在两处更新位置时，如何维护这里 scan 的高效查询。
3. Record 进入 RecordCache 和失效，驱逐部分的高效 Admission Policy。比如 update 之后, 以及 insert 导致页面分裂 SMO 之后 RecordCache 中信息过时的高效通知机制。

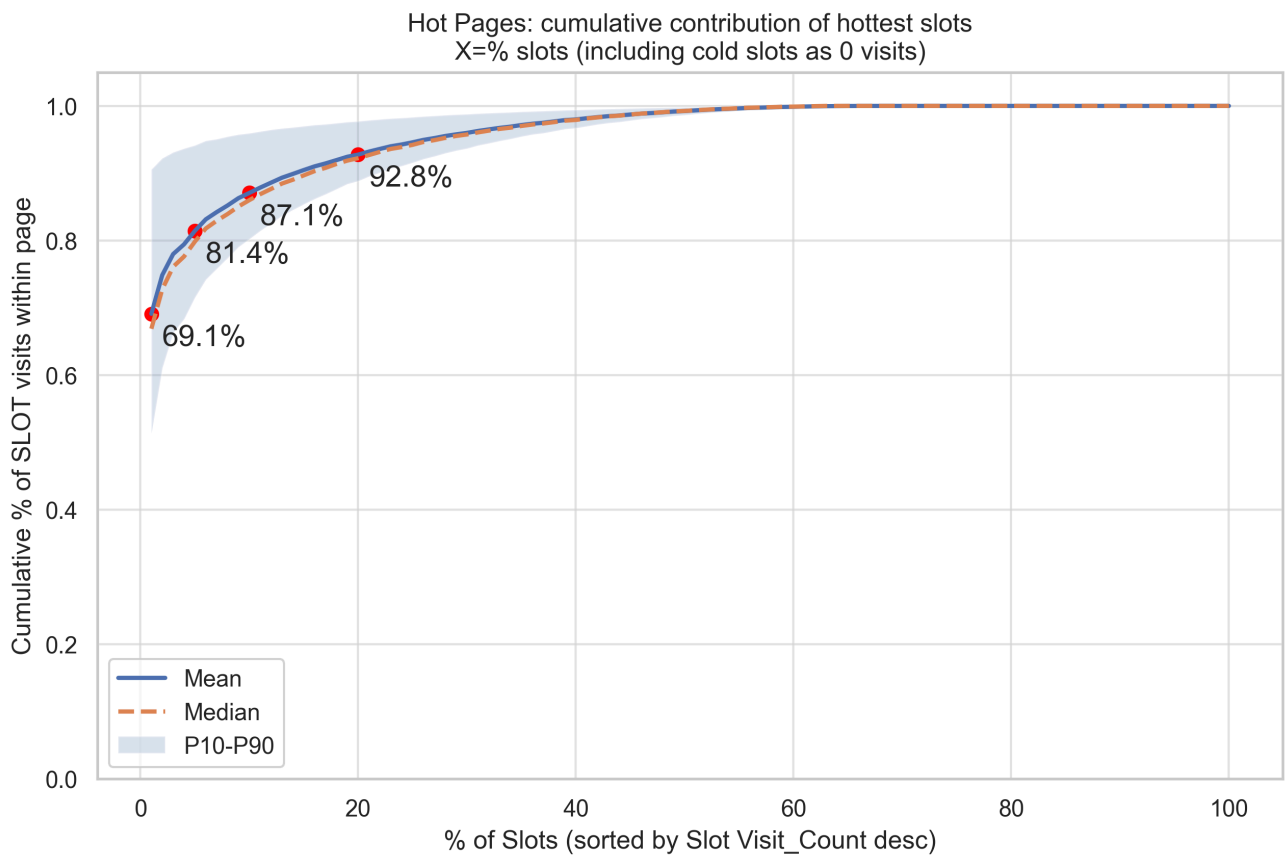
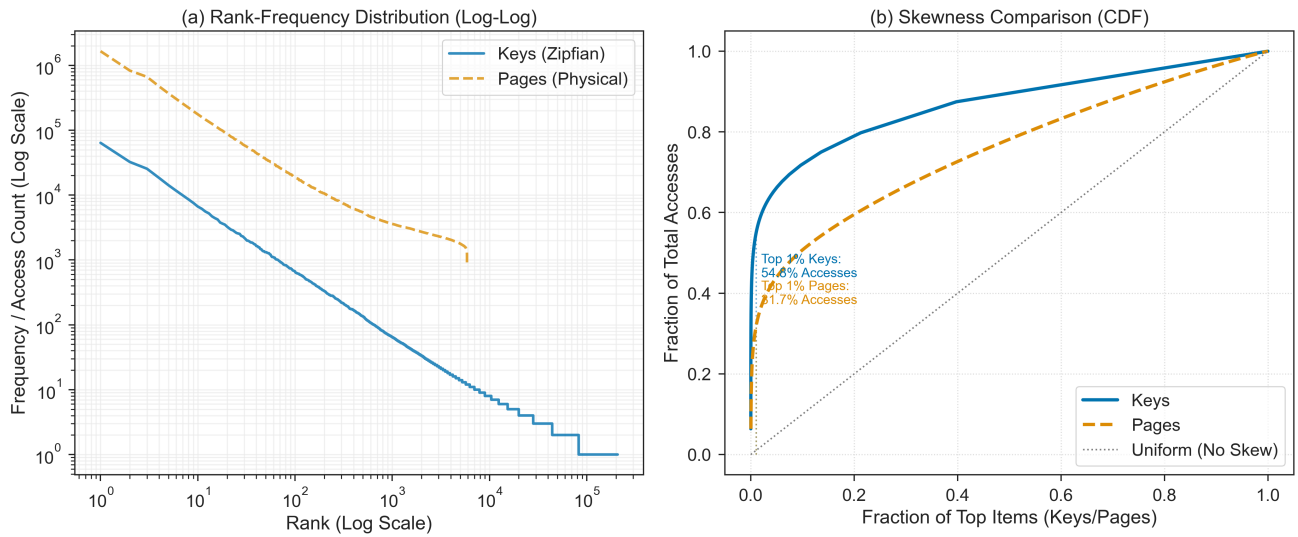
Motivation



Page Hotness vs. Slot Utilization



(16 K 页面情况):



我们跑了一个 Motivation 的实验，在 ScrambleZipfian 分布的 key 访问情况下 (Zipfian=0.99)，模拟真实场景中热点并不是集中在 B+Tree 中连续的头部，而是分散在 Page 中各处。

从 Page 访问的角度来看，由于 Page 的访问频次基本上由 Page 上最热的 Record 撑起来，图 5(a) 显示出 Page 访问的热度分布也接近 Zipfian 分布。然而由于统计粒度的不匹配，图 5(b) 展示了 100000 次查询中，按照 key 为粒度，Top 1% 的 key 能够应对 54.8% 的查询请求，然而 Top 1% 的 Page 只能应对 48.2 % 的查询请求。除了以 Page 为粒度统计会缺失热点访问记录 6.6% 的统计请求外，我们发现短记录 (8 B key + 8 B value) vs 4 KB Page 的内存差距。在图 2 中发现，大部分 Page 上只有 37% 的 Slot 被访问，然而由于 B+Tree 的平

衡策略，原本理论上一个 Page 可以容纳 116 个 Slot, 实际只使用了其中 43 个 Slot, 最终实际只访问了 15 个 Slot。在热点模糊判断的情况下，存在巨大的内存空间浪费情况。

图 [2]从 Leaf Page 的使用情况来看，存储一个真实数据对 4 KB 页面的利用率：

- (1) Page Header 记录，用去 32 B，有效的 Data 数组是 4064 B
- (2) 支持 MVCC 的 Tuple 头和 In-Place Update 的版本链，其中 trx_id 和 trx_ts 作为 8 B 的记录，是元数据开销的大头，占据 21 B(per-Record)
- (3) Slot 记录 key_length, value_length, offset 和查找效率时匹配 key 的公共前缀，占据 10B (per-Record)
- (4) 为了 Update 不溢出，以及在预热运行一轮 Scan 之后，稳定的页面为了写入性能，避免频繁发生页面分裂，每个 Slot 的有效装填负载是 43 个 Slot，最大上限是: 86 个 Slot，有效装填比是: 50% (fill factor)
- (5) 最后根据实际访问情况，发现有效装填的 43 个 Slot 中，实际访问过的 Slot 数目(只要访问过一次就记入统计)是 16个，有效访问比例是: 37%(effective utilization)

Related Work

Bf-Tree[1] 的优点是 scan-friendly, 问题在于并不适合 CXL 分层管理，出于延迟的性能下降，我们有充分的理由将 Record-Cache 和 Range Index 做分离,而 Bf-Tree 的结构并不适合分离出的 RecordCache。

Anti-Caching[6] 和我们的场景不一致，Anti-Caching[6] 把 DRAM 作为 primary-storage，核心是把 cold tuples 驱逐到磁盘，类似操作系统虚拟内存来扩展内存空间，解决 in-memory 内存不够放下工作集的问题，核心解决的是驱逐哪些冷 Tuple。

我们的工作首先将数据加载到 CXL，然后经过两级过滤，是让热数据做更细的划分，把更热的数据推到 Tier1(DRAM)，核心是准入哪些热 Tuple。

Challenge

Challenge 1. Admission Policy (Cold Page, Hot Record) 和 Hot Page 的问题

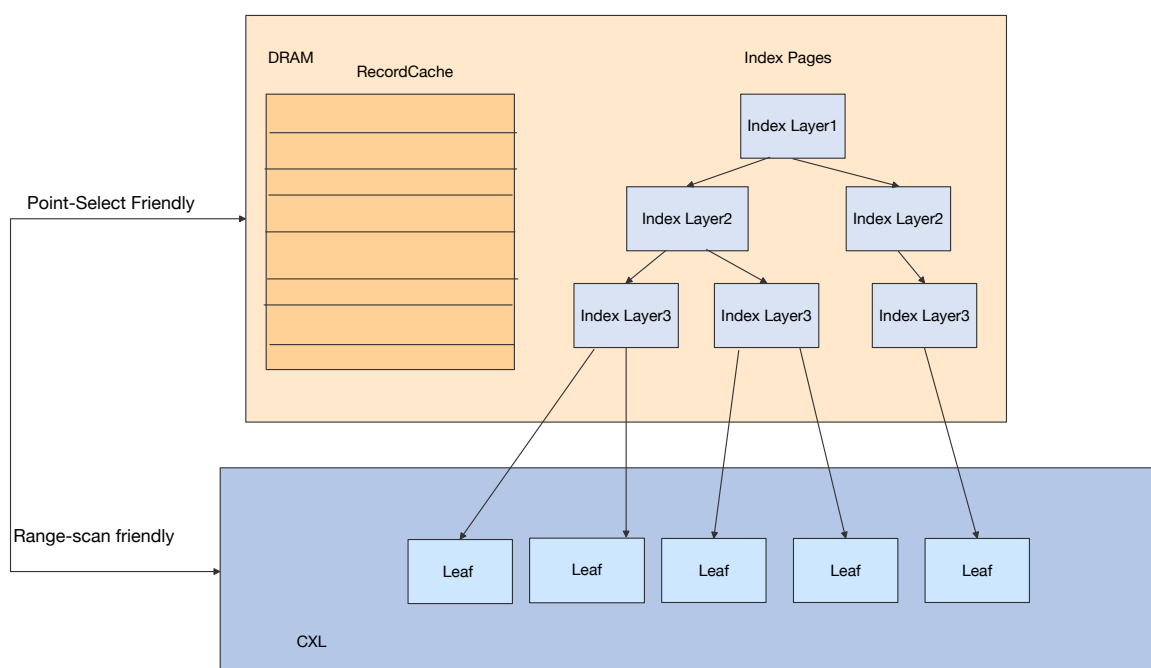
对于 Tier1 和 Tier2 的访问延迟差异，我们需要更明确地区分出 Hot Page 和 Hot Record。

核心统计的一个难点：

最简单的方法是维护线程局部变量，对每一个 page 上不同的 slot, 维护每个 page 上不同 slot_id 的访问次数，但这个具有很大的数据开销。

Challenge 2. 高效的 scan 操作设计

RecordCache 本身是服务于 point-select 这样的无序索引，CXL 层的 B-Tree 是服务于 scan 这样的有序范围索引。RecordCache 本身对 scan 的支持很差，如何进行高效查询。[1] 中的做法是不分离 RecordCache，动态将下图 CXL Leaf 中的容量从 64 B 调整到 4 KB，但是这种设计不适合存在访问延迟差异的 Tier 1(DRAM) 和 Tier 2(CXL) 层次，出于分层访问延迟的考虑，我们有充分的依据做 RecordCache 和 Page-Sized Buffer Pool 的分离。



Challenge 3. RecordCache consistency issue 一致性问题 解决和 Eviction 逻辑

Page Split 导致的 SMO 和 update 操作执行过程中，旧 record 失效的高效通知机制。以及在 Write-back 写回模型下，dirty record 的 merge 和 eviction 时机。

Design

我们分别考虑了下面两种写策略的模型。RecordCache 本身适合 point-select 这样的无序点查，同时为了解决写性能下降的问题，我们在 RecordCache 中支持 in-place update 和

lookup 查询, scan 和 insert 操作则落在 CXL Buffer Pool 中的 range-index (B+Tree) 结构中。

对于 in-place update 的操作, 分别有 write-back 和 write-through 两种写回策略。而 Read-Only 需要考虑更新 CXL Buffer Pool 之后, RecordCache 里对应记录的失效通知机制。这个包含在查询执行的关键路径上。

我们的设计中存在这样的开关, 来应对 read-only 这样的查询请求, 和 write-back/write-through 的开关, 来应对混合 read-write 请求。

对于持久化存储和崩溃恢复的部分, 我们沿用 Page 粒度 WAL 的逻辑。[5] 中把 Record 作为 Atomic Record Store(ARS), 专门设计了一套日志机制来完成 Record 粒度的 Recoverable Update, 这里我们认为持久化存储和崩溃恢复与缓存管理是正交的工作, 留作 future work。

(Cold Page, Hot Record) vs Hot Page

设计遇到的第一个问题是, 如何识别最热的那部分数据, 以及从中提取出适合放到 RecordCache 中的 Record, 准确区分 (Cold Page, Hot Record) 和 Hot Page 两种分类, 按 DRAM vs CXL 容量划分热点资源。(这里的 Page 只针对 B+-Tree 结构中的 Leaf 节点, 索引页面不作为驱逐的对象)。

=> 我们的设计方案: 两个 count-min-sketch, 分别用于划分可以进入 DRAM 的 Page 候选者, 和可以进入 DRAM RecordCache 的 Record 候选者。

在传统的 Page-Level Eviction 中通常采用 second-chance 的方法, 维护 Buffer Pool 的驱逐逻辑, 问题在于 second-chance 无法得到 Page 的热度信息, Page_visit_count = 2 和 Page_visit_count = 20 在 second-chance 下无法区分,

然而, 直接在每个 page 上增加访问次数计数器并进行全局访问次数统计, 会导致 Page 上的计数器成为并发读写争用的热点, 一次读页面也会转换成对当前页面访问计数器的写操作。

[2] Memtis 中采用了访问频率直方图和指数移动平均来动态提供冷却信息, 能够提供精确控制的冷-温-热数据识别, 但是在 os 层面的工作, Memtis 的访问直方图依赖硬件性能计数器的访问, 不在关键路径上, 对每个地址维护 {address, page_access_count} 的统计在不影响前台性能的情况下是容易做到的。但 DB 层面的访问统计需要软件操作, 造成并发热点争用。

[3] Count-Min-Sketch 的方法是低开销的典范, 但是没法从访问次数中恢复出 page_id。在我们的工作中, 从 CXL Buffer Pool 识别适合进入 RecordCache 的 Page, 进行 Page 上的 Record Migration 是重要的一环。

我们认为这里用 12 个桶(bucket) 来记录页面访问频率的做法能够有效节省内存空间, 随着访问次数上限增加而动态分配桶的数目(上限 12 个)。

[4] 相较于 Top-k 问题来说, 我们的重点有别于准确识别 Top k 个大象流, 而是实现不同容量对应访问粒度的划分, 我们不需要精确知道具体是哪 k 个 page_id 在 page_access_count

$\geq X$, 而是挑选 `page_access_count  $\geq X$` 的可能候选者。

我们的做法是:

(baseline)

1. 维护一个 `page_id` 访问对应的 `count_page_min_sketch` 二维数组, 比如 `count_page_min_sketch[12][1024 * 16]`
2. 定期采样, 在预热过 1min 之后, 每次当前 `page_id` 的查询填充完 `count_page_min_sketch` 数组之后, 写入到对应 bucket 的计数里。

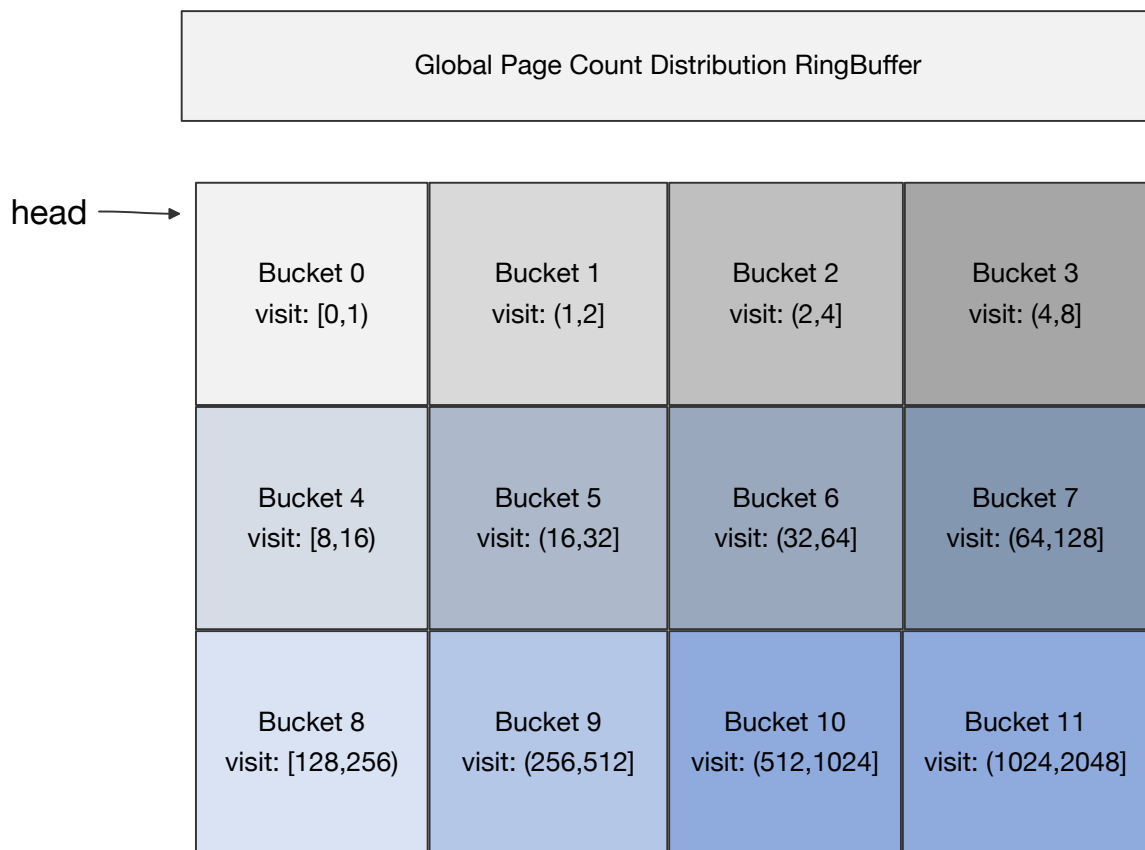
(这里维护全局的访问频率直方图, 目的是准确识别热点信息。但 `count_page_min_sketch` 本身是流式的, 如果直接将每次访问动态归类到某一个 bucket 0 中, 会出现同一个 `page_id` 先出现在 Bucket 2, 经过一段时间的访问, 又在 Bucket 3 中统计了一次的重复统计问题。

因此为了解决 `count_page_min_sketch` 的流式统计和 Global Page Count Distribution 是在某一个时刻的静态状态快照的问题, 我们采用 1024 大小的去重 set, 记录当前访问过程中出现的 1024 个不同 `page_id`, 读取 `count_page_min_sketch` 并填写对应 Bucket 信息, 维护一个局部访问频率直方图, 用局部分布代替整体, 局部访问频率直方图在 30s 后清空, 去重 set 也置空, 进入下一轮, 在当前时间窗口内保证没有重复统计)。

3. 桶对应有 30s 的超时老化机制, 每 30s 会把指针向后移动, head 指针永远记录访问次数在 $[0, 1)$ 的数据, Ring Buffer 的好处是 Aging 机制可以直接将访问计数减半, 对应只需要移动 head 指针。`count_page_min_sketch` 的老化采用 $\gg 1$ 的操作, 访问计数减半。
4. 根据当前的 bucket 建立一个横坐标为 12 个不同的访问区间, 纵坐标为不同 `page_id` 数目的直方图, 根据容量比例找到需要进入 RecordCache 的 Bucket 对应的访问次数, 比如对应访问次数在 2^x 之上。

(前 4 步都是低开销, 不涉及高频并发争抢的操作)

5. 动态处理, 当一个 `page_id` 请求到来时, 如果在 `count_page_min_sketch` 的基础上发现当前访问次数在 2^x 之上, 则标记该页面是可以进入 RecordCache 的 Candidate。



这些访问热点的区间对应的页面可以进入到下一轮 RecordCache 的统计中。

record-count-min-sketch

对于进入 RecordCache 中 record 的访问频率统计，采用 [3] count-min-sketch 的经典方法。我们默认维护一个 count_min[12][1024] 的二维数据，对于每次访问 record_id, 共有 12 个哈希函数 h0,h1,h2,..., h11统计出对应的位置, 比如 count_min[0][h0]++, ... count_min[11][h11]++, 最后的访问次数通过 min{count_min[0][h0], count_min[11][h11]} 得到。低开销的 record-level 访问次数统计是已经解决的问题，采用 [3] 的经典做法。

Two-Level Admission Logic

我们对于 CXL vs DRAM 的细粒度混合粒度缓存划分步骤如下:

1. 对当前访问的所有请求，在 CXL 中维护一个 Count-Page-Min-Sketch, 同时后台维护 12 个 Bucket 进行 Aging 的老化时机划分。当一条针对 page 的查询请求到来时，首先会有 12 个 哈希函数在对应的二维数组中记录 12 个访问信息。
后台线程负责定时扫描全表的信息，得到访问区间和访问次数的统计直方图🇮🇹。

2.

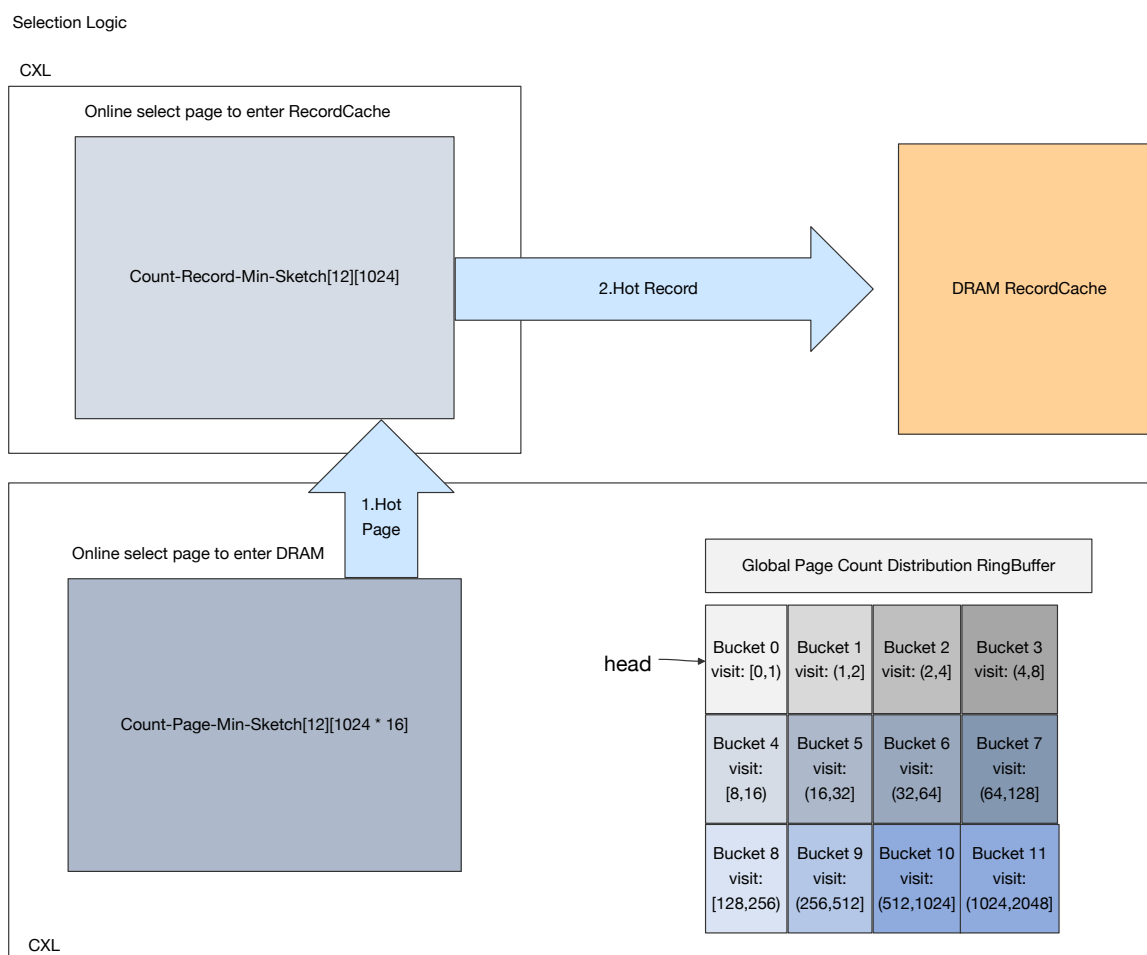
3. 新到来的请求执行在线操作，一旦通过 count-min-sketch 算法计算出访问频率超过符合 DRAM 缓存容量的阈值时，进入 DRAM

=> 第一层筛选，确保随着访问次数变化，进入 DRAM 的永远是最热的数据。

4. 对进入 DRAM 的 page 维护每个 page_id_record_id 的访问次数统计，从中筛选出倾斜的访问。区分出 Hot Page 和 (Cold Page, Hot Record) 的情况，Hot Record 进入 RecordCache

=> 第二层筛选，区分出 Hot Page 和 (Cold Page, Hot Record), 保证 RecordCache 针对倾斜访问的负载尽可能节省 DRAM 空间的使用，提高 lookup 查询效率。

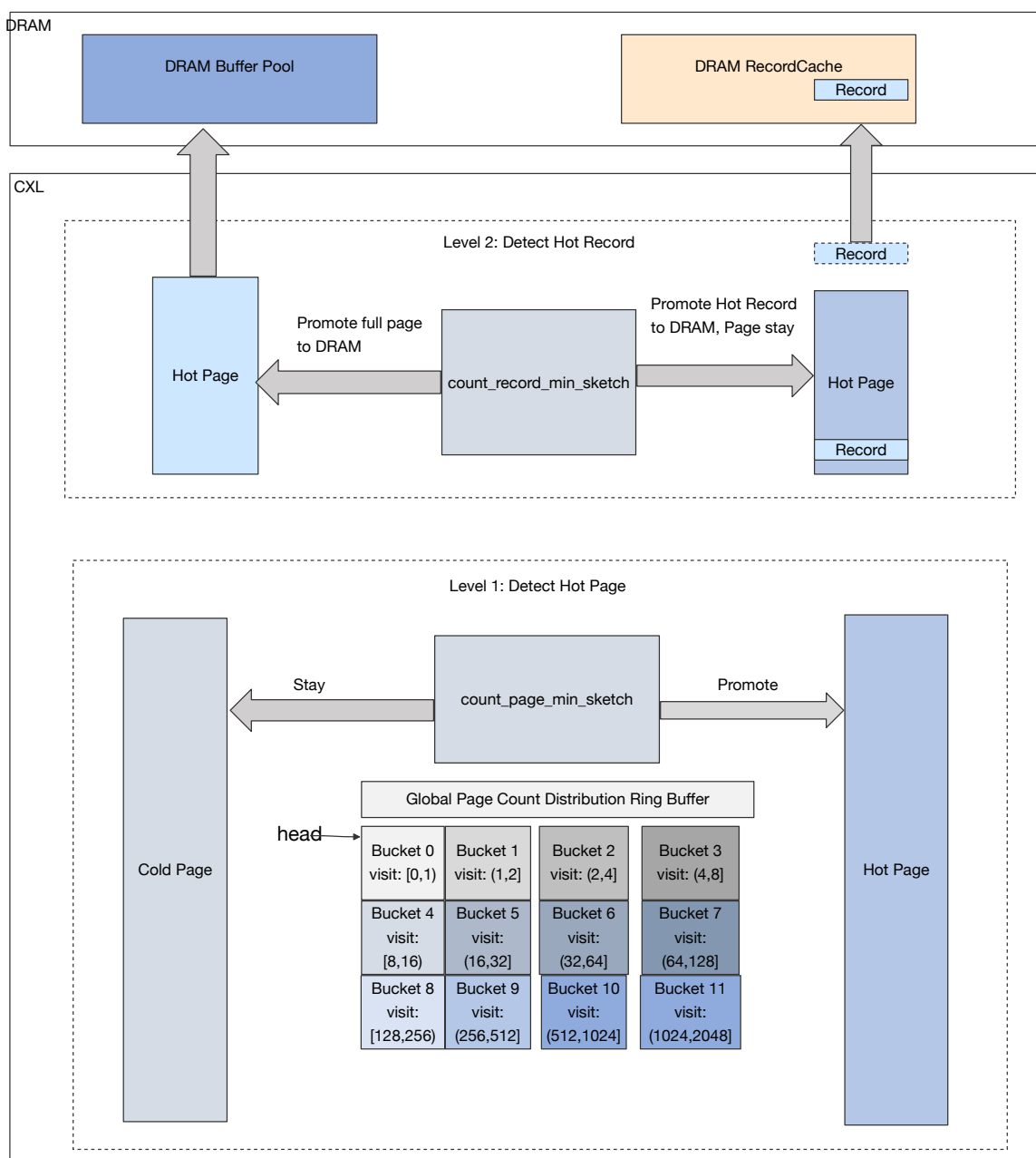
(这个设计的 trade-off 在于前几轮对 page 的访问流只累积在了 count_page_min_sketch 里，对于 record 的访问是在已经判断出 page_id 之后才开始记录的，丢失前几轮访问的信息)。

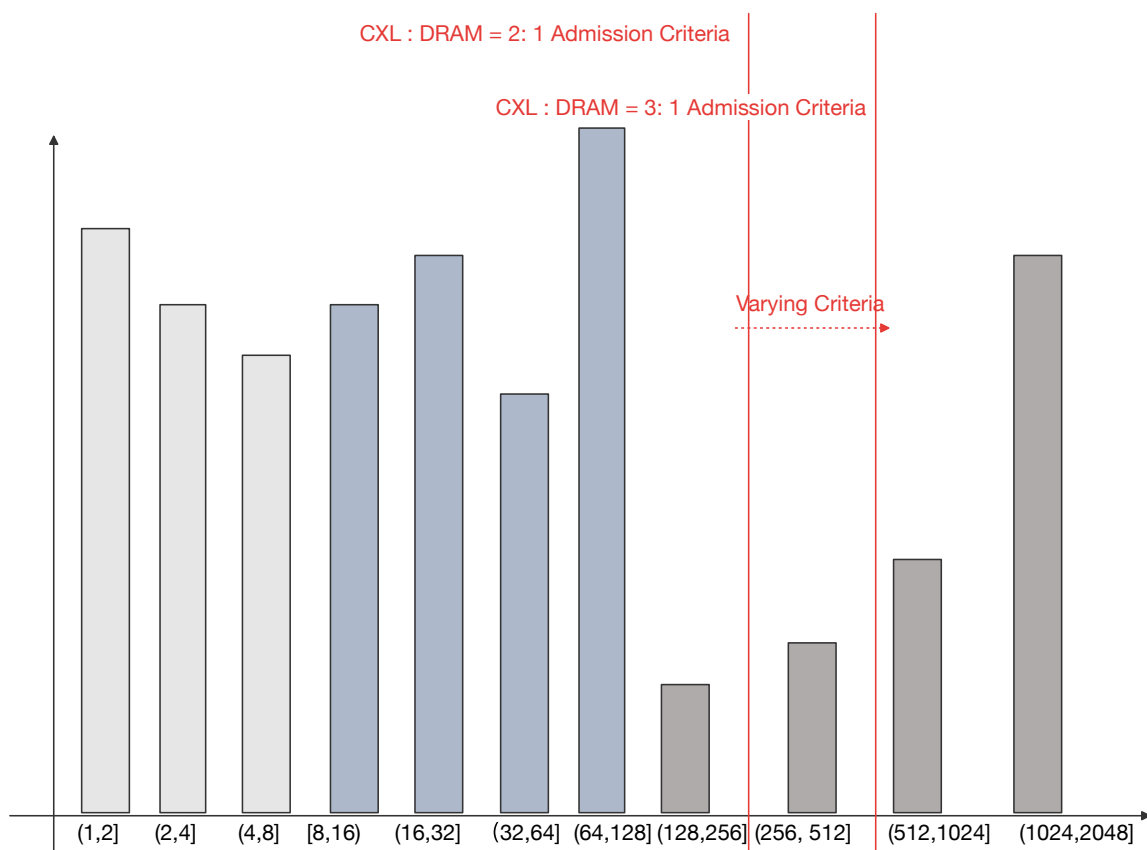


[5]Bw-Tree 中提到的 delta-record 附加在原先需要修改的页面上，减少 SSD 的写放大，转化成一连串的顺序写入。我们在分离了 RecordCache 和 B-Tree 叶子节点之后，对 RecordCache 的 Update 操作可以达到类似的效果。

我们提供下面两类写模型 write-back 和 write-through, 以及只读模式的 Read-Only 开关, 后续测试不同写模式下 DRAM(Tier1) - CXL(Tier2) - SSD(Tier3) 系统下 RecordCache 服务于 lookup/scan/update/insert 的性能表现。

其中展示 RecordCacheEntry, RecordCache 的结构, operation 和 eviction 执行流程。





在进入 RecordCache 的设计结构之前，对整体 Page 和 Record 的所有权做一个说明。

(1) 首先是在 CXL 中进行第一层 Hot Page 的检测，这里能够严格区分出对当前 Page 访问 20 次和访问 2 次的热度区别，通过局部访问次数直方图筛选出适合进入 DRAM 的 page_id，列为候选的 page_candidate。

(2) 被筛选出的 page_id 立刻进入 count_record_min_sketch 第二层的热度判断，检测当前页面上各个 record 的访问比例。

(2.1) 如果发现是均匀访问，则说明整个页面都很热，直接把完整的页面 promote 到 DRAM Buffer Pool 中。

(2.2) 如果发现页面访问具有明显的倾斜性，比如 20 次访问当前 page，有超过 10 次都是对同一个 record 的读写，则把对应的 record 移入 DRAM RecordCache，页面继续保留在 CXL 中。

在这里每个 page 只可能存在于一个地方，要么位于 DRAM Buffer Pool 中，要么位于 CXL Buffer Pool 中。我们的设计复用了 LeanStore[9] 的 Pointer-Swizzling，page_id 可以快速对应到 DRAM Address 或者 CXL Address。

对于 Record 来说，Record 可以同时存在于 DRAM RecordCache 和 CXL Buffer Pool 中，但是已经被判定为倾斜访问的 record 不会同时出现在 DRAM Buffer Pool 和 DRAM RecordCache 中。

在 RecordCache 的读写流程设计中，只需要处理 RecordCache 和 CXL Buffer Pool Page 上的一致性，DRAM Buffer Pool 的访问逻辑兼容一般的执行流程。

我们的读写流程图一致性设计中，只包含 Adapter Layer, DRAM RecordCache, CXL Buffer Pool, DRAM Index Page, SSD 这 5 个部分。

Consistency Model

为了充分测试 CXL 扩展内存场景下的读写性能问题，我们对 RecordCache 有下列三种定位: Read-Only, Write-Through 和 Write-Back。

作为 Hash 表结构的 RecordCache, 本身对点查的性能加速极大。从写延迟的角度考虑，CXL 的写延迟理论是 200ns, DRAM 的写延迟理论是 100 ns，在异步写 redo log 提交事务的场景下。我们将 update 的操作吸收在 DRAM 里，后台维护一个异步线程专门处理将这些 dirty record 写回并合并到 CXL Page 上。

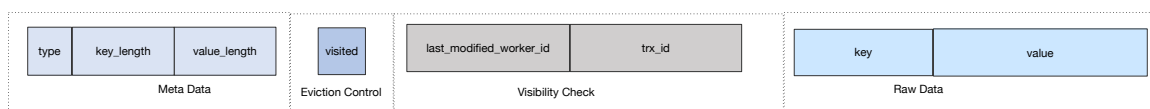
通过实验验证 write-back 和 write-through 两种写回模式下，RecordCache 的读写加速效果。

RecordCache 的结构是 Key 作为哈希的键，当前 RecordCacheEntry 的地址作为哈希的值。

Read-Only

整个 RecordCache 作为只读缓存，不执行 update 操作，核心是处理 CXL BufferPool 上更新 Record 之后，DRAM 中 RecordCache 的失效通知机制，以及 admission 和 eviction 的判断逻辑。

1. RecordCacheEntry



Read-Only 模式下的 RecordCacheEntry 格式是最基础的，占用空间最少的一个结构。Write-through 和 Write-Back 的部分会在此基础上叠加。

需要考虑核心的三个要素:

1. Eviction 字段如何做统计

2. CXL Buffer 执行 Update 之后, 如何失效掉现在的 Record
3. CXL Buffer Pool 执行 Insert 导致产生页面结构分裂(SMO) 后, RecordCacheEntry 是否需要修改。

目前的字段大小统计: 由 4 个部分组成: Meta Data, Eviction Control, Visibility Check 和 Raw Data

Meta Data:

atomic type(std::atomic 3 bit):

000 表示 Read-Only 模式,

001 表示 Write-Through 模式,

010 表示 Write-Back 模式,

011 表示逻辑删除, 但是物理上数据仍然存在哈希表中, 等待物理删除。之后的线程遇到 011 则表示这是一个无效记录。

100 表示数据已经从哈希表中移除, 等待 Eviction 线程执行物理内存释放。

111 表示 Promote 线程占位。

Type 字段是状态转换的关键, 切换时需要 std::memory_order_release 和 std::memory_order_acquire。

key_length(2 Byte): 支持 Varchar 类型的变长键

value_length(2 Byte): 支持 Varchar 类型的变长 String 字段

Eviction Control:

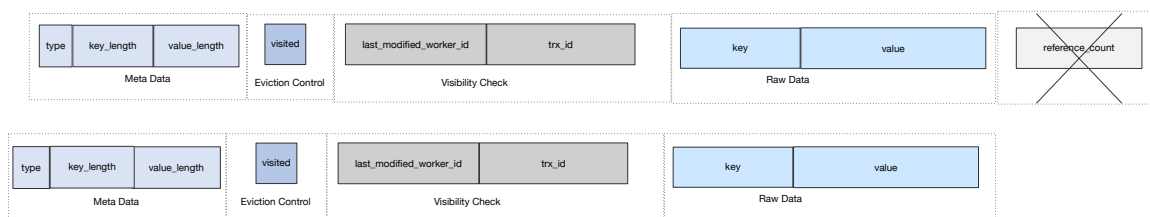
visited(std::atomic 1 Bit): 集成 SIEVE 的驱逐逻辑, 每次驱逐之前把 visited 设置为 1

Visibility Check:

last_modified_worker_id(2 Byte): 最后一次修改此事务的 worker_id

trx_ts(8 Byte): 最后修改这条 Record 记录的事务 id, 兼容已提交事务的提交时间, 和未提交事务的开始时间, 根据最高位的 1 Bit 判断是否事务已提交。

Invalidation metadata

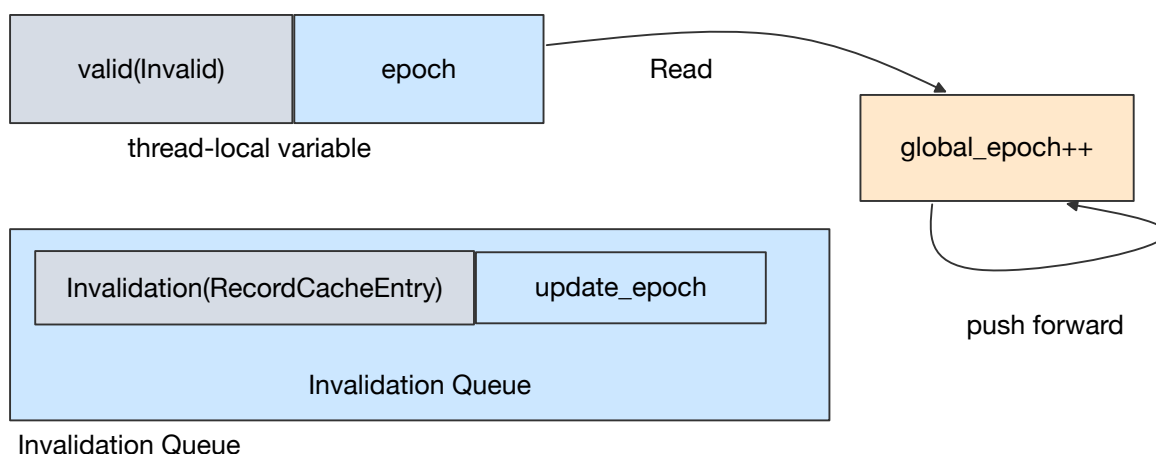


对 RecordCacheEntry 做驱逐时，最简单的访问控制是reference_count。Reference_count 的问题是，每次读访问都会变成两次原子访问, atomic_add 和 atomic decrease, 读访问变成对 1 Byte reference_count 的两次原子写操作。

[12] Silo 的工作优化了只读数据的访问: 避免将只读变成共享内存记录的写访问。我们将其 中 Epoch 的思想纳入这里 DRAM RecordCache 的 Invalidation 方案中。

引入线程局部变量 thread_local_variable {epoch_active, epoch}; 以及一个全局变量 global_variable global_epoch;

Epoch 是 RCU 算法的变种，从 RecordCacheEntry 的每个字段维护的变量，变成线程局部 读写的变量，尽可能消除 Read-Only RecordCacheEntry 上的写操作。



Invalidation Flow

Invalidation 阶段的执行过程，现在有 Thread A(lookup) 和 Thread B(update), 核心是 lookup 和 update 在并发执行过程中的一致性问题。后台有一个专门的线程 Forward_epoch

1. Thread A 先执行，首先读到 global_epoch = 0 的全局变量，把当前 global_epoch 的值 写入自己的线程局部变量, thread_A_epoch_active = True, thread_A_epoch = 0;

2. Thread A 执行 lookup, 但存在并发的先后顺序

(这中间 Forward_epoch 会定期更新 global_epoch, global_epoch++)

3. Thread B 读到 global_epoch = 1 的全局变量, 把当前 global_epoch 的值写入自己的线程局部变量, thread_B_epoch_active = True, thread_B_epoch = 1;
4. Thread B 立刻修改对应 RecordCacheEntry 的字段, 把 Type 设置为 011 标记逻辑删除。同时把 RecordCacheEntry 加入 Invalidation Queue, 附上 update_epoch 信息。
5. Invalidation, Thread B 希望 Invalidation, 此时需要轮询所有线程, 判断是否有 thread_local_epoch < update_epoch 的 thread_epoch_active = True, 如果所有的读事务已经终止, 设置为 False, 则能够执行 Invalidation 的物理删除操作, 物理删除改记录。同时 Invalidation 线程需要负责从哈希表中彻底删除 key 的功能, 这里根据 Hash(key) 找到对应的 Bucket, 获取哈希表结构的分段锁, 由于 Epoch 机制严格确认当前记录已经失效, 可以安全移除。

Eviction metadata

从内存开销的角度考虑, Eviction 有两种常见的做法

- (1) [10] Redis 的做法, 每一个 RecordCacheEntry 留出 3 Byte 的字段作为 LRU 或 LFU 的得分信息统计。

在驱逐过程中, 维护一个只有 6 个元素的 Eviction Pool, 采样得到 6 个元素之后驱逐其中得分最高的对象。

- (2) [13] SIEVE 的做法, 每个 RecordCacheEntry 只需要维护 1 Bit 的 visited 信息(之后的多次访问只需要统计一次, 比如对同一个 RecordCacheEntry 的 100 次访问, 只有第 1 次会设置这个状态位)。但是需要维护一个 FIFO 队列, 相当于每个 RecordCacheEntry 对应 8 Byte(FIFO 队列的开销)。

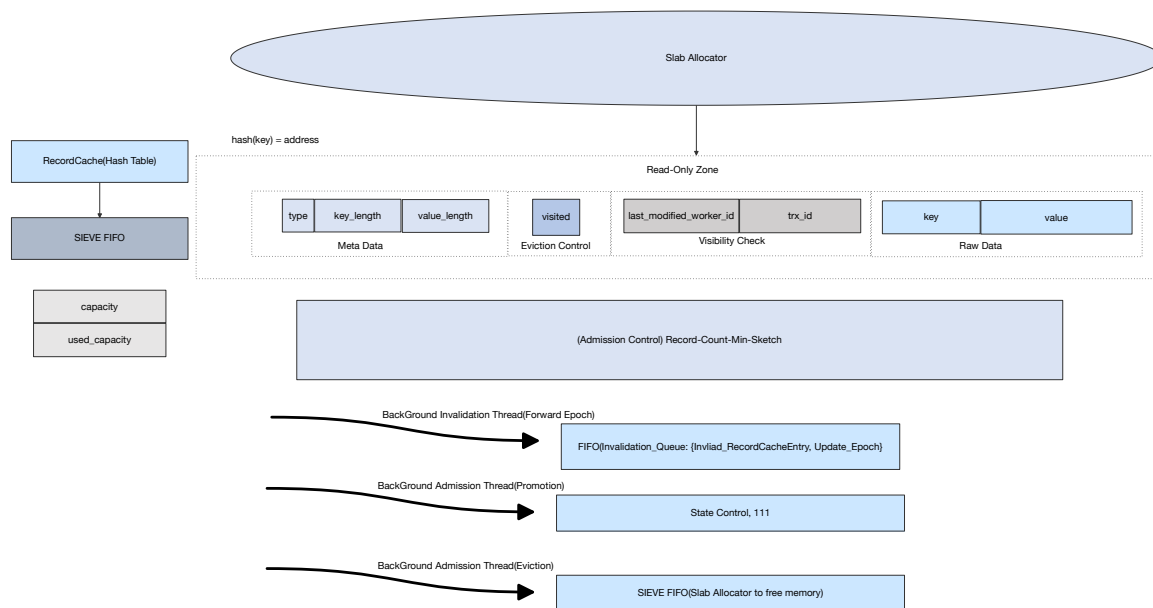
我们目前选择 (2) 的方法, 因为之前 Invalidation 字段的设计已经在极力避免只读操作对 RecordCacheEntry 的写入修改, Redis 做法的 3 Byte 需要将每一次读访问变成写访问。

如果 (2) 效果不佳再回退到 (1) 中增加额外的 3 Byte 作为分数字段的统计标准。

SIEVE 的 Eviction 阶段也需要 Epoch 的机制来取代引用计数对当前线程访问数据的保护。

2. RecordCache

RecordCache 本身是 Hash 表结构, 此外维护一个 Read-Only 字段的 SIEVE FIFO Eviction 队列。



1. Hash(key)= address(RecordCacheEntry*), Slab Allocator 保证了在物理删除之前，地址是保持不变的。
2. Hash 表采用链地址法，失效 key 时直接获取 bucket 锁。
3. Forward_epoch 线程承担两个功能:
 - (1) 周期性更新 global_epoch, 推进 epoch 进展,
 - (2) Invalidation, 直接执行 epoch 失效的哈希表记录的物理删除，调用 RecordCache bucket 锁执行 remove(key, entry_pointer), 同时释放当前 RecordCacheEntry 从 slab allocator 中申请得到的内存空间。
4. 本身存储引擎支持 multi-writer, 但是如果让每一个涉及 lookup 的线程自己处理 promotion 的逻辑，会严重拖慢读事务的执行提交，所以这里采用后台线程做 promotion。Multi-worker 对应 Single-promoter 可能会导致 cache 准入不及时，造成瓶颈。目前的方案是 RecordCache 做分片，每个分片对应一个 promote 后台线程，分片数目动态调整。
5. Read-Only 线程模型描述: Multi-worker 执行 CXL BufferPool Update, Single-writer 执行 RecordCache Insert。

Memory Allocator

这里维护一个基本的内存分配模块 Slab Allocator, [14] Memcached 常用的 Slab Allocator 方法。其中单次申请的内存大小是 2 MB, 每次分配的 size 按照 8 Byte 为步长做分类。根据运行时 Record 的大小动态维护不同 size 的 free_list, 消除外部碎片。同时 8 Byte 的步长在分配的同时尽可能减少内部碎片。

整体上有 Global Allocator, 分配一个 2 MB 的大页，并负责维护不同 size 的 free_list。

对于当前设置的多个 worker 线程唤醒一个 promote 线程的操作，多个 RecordCache 拥有不同的数据分片，每个分片具有一个 promote 线程。

3. lookup/scan/insert/update/Promote workflow

Read-Only 模式下，Record 被 Promtion 的情况只可能是 Lookup 线程的多次访问导致的，Update 线程涉及到写操作，Invalidation 操作由 Forward_epoch 线程完成，RecordCache 的 insert 操作由 Promote 线程完成。

(核心的执行流程总结:

1. 只有 Lookup 线程会触发 Promote 操作
2. Update 线程在执行 Invalidate 操作之前，先设置 RecordCacheEntry 中 Type 字段的标志位，再执行 CXL BufferPool 的实际 update 操作。
3. 高效的 RecordCacheEntry 失效通知机制由线程局部变量和 Forward_epoch 线程完成，我们将每个字段的 reference_count 引用计数写变成线程局部变量写，减少了共享数据的写操作热点竞争。)

核心的线程: Lookup(trigger new), Update(trigger delete), Promote(new), Forward_epoch(delete)

Scan

scan 直接走 CXL BufferPool 中的 Page 进行 scan 查询，没有额外的操作。

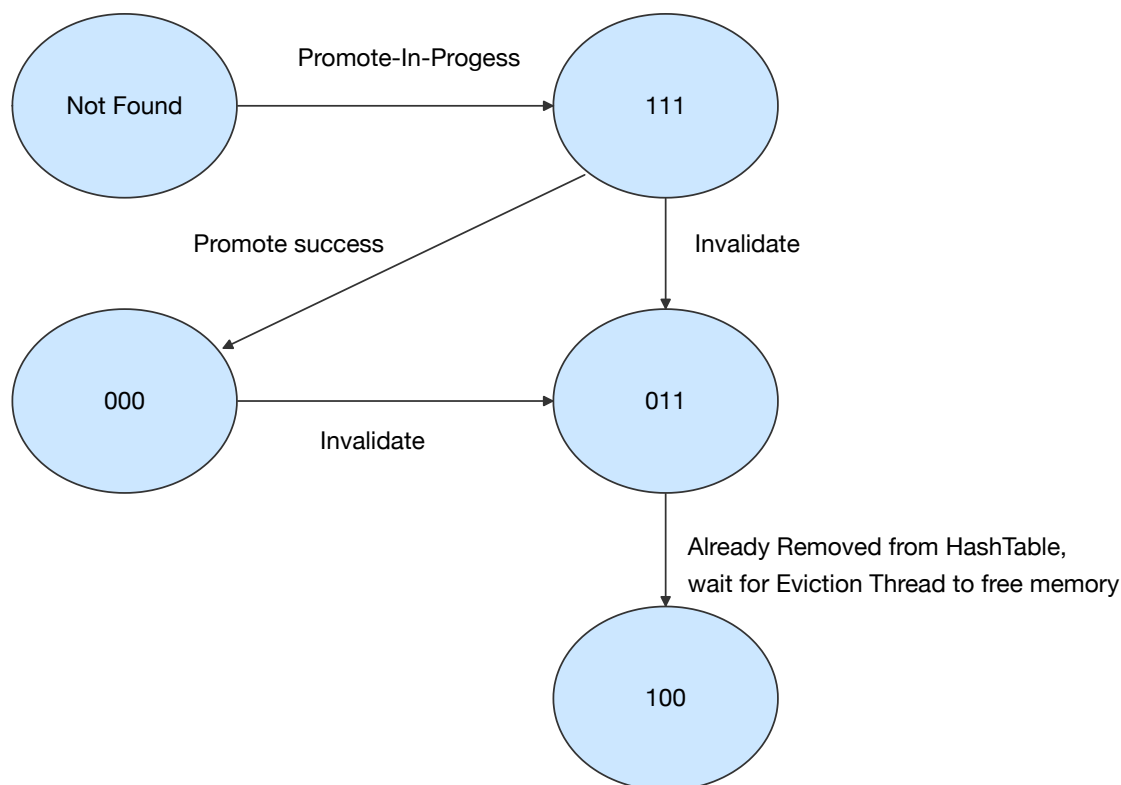
Insert

Insert 直接在原先的 CXL BufferPool Page 上进行分裂，由于 RecordCacheEntry 字段中没有存储 cxl_record_pointer, 且 Promote 逻辑传递的 cxl_memory_address 配合 Page_id 和 offset, 不需要 Insert 逻辑特别通知。

State Machine Definitions

本节回顾下面 4 类操作在不同线程执行阶段导致的冲突问题，Lookup Thread, Update Thread, Promote Thread, Forward_Epoch Thread。

在 Read-Only RecordCacheEntry 字段中不设置锁控制，依赖 RecordCache Hash 表结构的 bucket 锁来控制 RecordCacheEntry 的 Admission。状态机的核心转换是 Type 字段。



1. Lookup Thread

Lookup 线程的执行流程:

1. 读取 Global_epoch, 记录下当前线程局部变量状态 {active, current_epoch = lookup_global_epoch}
2. 判断 RecordCache 中是否有当前记录, Hash(key)

根据 Hash(key) 的返回值, 有如下情况:

a. Type = 000, Hit, 直接返回

b. Type = 011, 逻辑删除, 读 CXL Buffer Pool, 将对应 Count-Min-Sketch 的位置访问次数 +1, 判断是否超过阈值, 超过则唤醒 Promote 线程。给 Promote 线程发送 CXL Buffer Pool 中 RecordCacheEntry 地址的相关信息, 比如 (cxl_address, offset), 方便 Promote 线程的直接内存访问。

c. Type = 111, 已经有线程在处理 Promote, 读 CXL BufferPool, 将对应 Count-Min-Sketch 的位置访问次数 +1。

d. Not Found, 读 CXL Buffer Pool, 将对应 Count-Min-Sketch 的位置访问次数 +1, 判断是否超过阈值, 超过则唤醒 Promote 线程。给 Promote 线程发送 CXL Buffer Pool 中 RecordCacheEntry 地址的相关信息, 比如 (cxl_address, offset), 方便 Promote 线程的直接内存访问。

3. 退出之前将当前线程局部变量的 active 设置为 false。

2. Update Thread

Update 线程的执行流程:

1. 读取 Global_epoch, 记录下当前线程的状态 {active, current_epoch = update_global_epoch}
2. 判断 RecordCache 中是否有当前记录, Hash(key)

根据 Hash(key) 的返回值, 有如下情况:

a. Type = 000, 立刻调用 CAS(Compare-and-Set), 将 Hash(key) 对应的 Type 字段设置为 011, 标记为逻辑删除。

b. Type = 111, 不能释放, 因为 Promote 线程还在使用。这里仅设置 Type 字段为 011, 由 Promote 线程完成释放。

c. Type = 011, 已经有线程标记过逻辑删除, 只需要更新 CXL BufferPool。

d. Not Found, 不需要处理 RecordCache 相关逻辑, 只需要更新 CXL BufferPool。

3. 对于 2.1 中的情况, Update 线程负责将对应逻辑删除的记录放入 Invalidation_Queue 其中失效队列是分片的, 可以实现成 FIFO 队列, 将 {Invalidation_RecordCacheEntry, update_global_epoch} 放入 Invalidation_Queue。
4. 退出之前将当前线程局部变量的 active 设置为 false。

3. Promote Thread(ABA Issue)

Promote 线程的执行流程:

1. 接收到 Lookup 线程的唤醒信号, 和对应的 Record 信息。
2. 在 RecordCache 中查找是否有对应的项。
 - a. Type = 000, 已经完成, 直接返回。
 - b. Type = 111, 直接返回(已经在相同记录上被重复唤醒过)。
 - c. Type = 011, 不复用该字段, 重新申请, 插入 Bucket 链表头部。

获取当前 RecordCache 的 bucket 锁, Promote 线程从 Slab Allocator 中申请对应 RecordCacheEntry size 的空闲内存, 初始化时 Type 字段设置为 111, 表示由 Promote 线程置位控制, 插入当前 RecordCacheEntry 进 RecordCache。插入完成之后解锁。

d. Not Found, 重新申请, 插入 Bucket 链表头部。

获取当前 RecordCache 的 bucket 锁, Promote 线程从 Slab Allocator 中申请对应 RecordCacheEntry size 的空闲内存, 初始化时 Type 字段设置为 111, 表示由 Promote 线程置位控制, 插入当前 RecordCacheEntry 进 RecordCache。插入完成之后解锁。

3. Promote 线程在 CXL BufferPool 中读取记录, 写到自己的线程局部缓存中。

4. 获取 HashTable bucket 锁, 先确认 Type 字段仍然是 111, 没有发生 Promote 线程读(A) -> Update 线程改 (B) -> Promote 线程写(A) 这样的并发问题。

- a. 如果 Type 字段被 Update 线程失效为 011, 则由于 Promote 线程已经持有 bucket 锁, 在当前持锁状态将该 RecordCacheEntry 从 HashTable 中执行 HashTable.remove(key, entry_pointer), 同时调用 Slab Allocator (拥有 key_length, value_length) 信息, 释放内存资源。最后解锁 bucket, 并释放本地线程缓存的当前记录资源。
- b. 如果 Type 字段仍然是 111, (double-check) 通过, 将数据从局部缓存中拷贝到对应字段, 新 Record 准入操作完成, Type 字段变成 000。同时将新 Record 加入 SIEVE Queue。

4. Forward_Epoch Thread

Forward_epoch 线程的执行流程:

1. 周期性更新全局变量 global_epoch, 执行推进。
2. Invalidation_Queue 中一旦有内容, 根据当前 Invalidation_Queue 中的

{Invalid_RecordCacheEntry, update_epoch} 读出 update_epoch,

轮询所有当前分片对应的 {epoch_active_state, current_epoch}, 检查所有小于 update_epoch 的 epoch 状态, 如果还有 active 的情况, 说明逻辑删除的 RecordCacheEntry 仍然在被 Lookup 线程使用, 不能执行物理删除。否则执行物理删除。

3. 物理删除涉及两个操作:

- a. 获取 RecordCache 的 bucket 锁, 执行 hashtable.remove(key, entry_pointer), 从 RecordCache 中删除 key。
- b. 同时把 entry_pointer 对应的 RecordCacheEntry Type 字段设置为 100, 表示已经从哈希表中移除。

5. SIEVE Eviction Thread

关于 Forward_Epoch Thread 和 SIEVE Eviction Thread 交互的过程, 如果直接由 Forward_Epoch 执行物理的 RecordCacheEntry* 对应内存空间的删除。在后续 Eviction 的过程中, Eviction Thread 没有得到任何关于 FIFO 队列中存在的 RecordCache* 内存失效, 已经被 Slab Allocator 回收的消息。在后续根据指针访问遍历 visited 字段时会产生段错误。

这里把 Forward_EPOCH Thread 的功能划分, Forward_EPOCH Thread 做 Invalidation 的判断和从哈希表中 hashtable.remove(key, entry_pointer), 把 Type 字段设置为 100, 交由 Eviction 来做 Slab Allocator 的内存释放。

SIEVE Eviction 线程的执行流程:

1. 从 FIFO 队列中取出 entry_pointer
2. 检查 entry_pointer 的 Type 信息。
 - a. Type: 000, hand 指针对应 visited = 0,

此时是由于长期无访问导致的驱逐, 获取 HashTable 的 Bucket 锁, 在拿到锁之后 Double-Check Type 字段, 在 Type = 000 的情况下, 执行 hashtable.remove(key, entry_pointer), 调用 Slab Allocator 执行内存释放。

- b. Type: 100,

此时是由于 Update 导致记录失效, 直接调用 Slab Allocator 执行内存释放。

- c. Type: 011/111: 当前记录属于中间状态, 将 entry_pointer 重新插入 SIEVE FIFO 队列的尾部。

6. Summary

Thread\Type	000(Read-Only)	011(逻辑删除)	111(正在 Promote 中)	Not Found
Lookup	Hit: 直接返回	Miss:读 CXL , 访问计数++,可能触发 Promote	Miss:读 CXL , 访问计数++,不触发 Promote(去重)	Miss:读 CXL, 计数++,触发 Promote
Update	Invalidate: CAS 变成 011, 记录加入 Invalidation Queue	Ignore: 只更新 CXL。	Invalidate: CAS 改成 011, 不加入 Invalidation_Queue	Ignore: 只更新 CXL
Promote	Abort: 已存在, 直接返回	Proceed: Miss, 申请分配新节点 (Type=111)插入头部, 不处理 011。	Abort: 已存在, 直接返回	Proceed: 申请分配新节点 (Type=111)插入头部

000 -> 011: Update 线程负责送入 Invalidation_Queue,由 Forward_EPOCH 线程负责清理哈希表。

011 -> 100: Forward_Epoch 线程清理哈希表完毕，由 SIEVE_Eviction 线程负责释放内存。

111 -> 011: Update 线程只通知，Promote 线程负责清理 Promote 过程中由于并发或者失效导致 Promote 失败的那部分内存。

Write-Through

Write-Through 模型本质上是同步更新，每次写 CXL BufferPool 都同时写 RecordCache。这里的崩溃恢复和持久化在原有的 Page 粒度的 WAL 机制上自然满足。牺牲写 CXL 的性能下降来换取一致性保证，无需复杂的机制，作为写性能的 baseline 呈现。

Write-Through 本身提出的 Motivation 是，探讨在 CXL 硬件特点(~200ns) 延迟的情况下，RecordCache 作为 Write-Through 的缓存还是作为 Write-Back 的 RecordStore 对整体的写性能更具收益。在近内存的层次下(Near-Memory), 软件上的复杂度对写性能的影响。

在原先 Read-Only RecordCache 的设计逻辑里，引入 Epoch-based Invalidation 本质是为了减少单次查询过程中 heavy 的写原子操作。在 Read-Only 的逻辑中，一条记录从被更新失效到重新写入经历了下面的生命周期：

Invalidate -> Wait -> Clean -> Miss -> Promote

(更新失效周期)

[Update 通知记录失效] -> [Update 放入 Invalidation_queue] -> [Forward_Epoch 反复轮询读线程判断是否可以从哈希表中删除] -> [Forward_Epoch 验证成功，从哈希表中删除] -> [(内存回收): Eviction 队列等待 Pop 出的记录，释放内存]

(Update 后准入周期)

[Lookup 多次查询 Miss 发现适合准入] -> [Promote 线程置位，准备插入 RecordCache] -> [Promote 线程进行准入判断，是否已经被 Invalid] -> [准入判断成功，申请新内存插入 RecordCache]

对于 Heavy-read-write-mixed 的负载来说，单条记录经历频繁的 Update，生命周期过长，容易产生 cache Thrashing，支持 Write-Through 能够应对这种场景下的访问。

Read-Only 对 Hot Read 友好，Write-Through 对 Hot Mutable 友好。

Write-Through 的部分在 Read-Only 的基础上扩展，复用 Slab Allocator, Admission, Eviction 的部分。

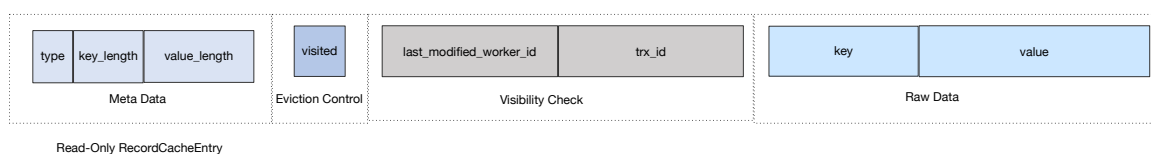
Write-Through 设计部分需要考虑的核心问题：

1. 原先的 Page-Sized BufferPool 更新的核心逻辑是 In-Place Update 和复制旧记录追加到版本链上。

本身在 RecordCache 中维护版本链的成本高，RecordCache 的更新版本选择问题。

- RecordCacheEntry 锁结构的设计和两级存储的锁顺序。
 - 同时更新 CXL BufferPool 和 RecordCache 时如何快速索引两者，同步过程中尽量减少查找时间。
- (比如直接在 RecordCacheEntry 中维护指向 CXL BufferPool 中 Record 的指针，但是指针会因为 SMO 等操作产生移动，原先的 Record 指针失效的先后顺序)。
- 执行情况的中间状态回滚机制，比如 CXL BufferPool 更新成功，但 RecordCache 更新失败，回滚通知。
 - Lookup 和 Update 阶段的冲突控制。

1. RecordCacheEntry



回顾原先的 Read-Only RecordCacheEntry, 其中 Read-Only 部分用于状态转换逻辑判断的 Type 字段可以总结如下:

001: Write-Through

011: Update 标记当前 RecordCache 中记录被判断为逻辑失效

100: 表示记录已经从哈希表中删除，等待 Eviction 线程做物理释放

111: Promote 线程正在执行 Promote 操作



在 Read-Only RecordCacheEntry 的基础上，增加下列字段:

Quick Index to CXL:

Swip(Page id or BufferFrame*: 8 Byte):

快速索引，RecordCache 中维护指向 CXL BufferPool 中 BufferFrame 的指针，由于 CXL Buffer Pool 可能产生页面换出到 SSD，Swip 本身可以表示 Page id 和 BufferFrame 指针的 union。

Slot_id(2 Byte):

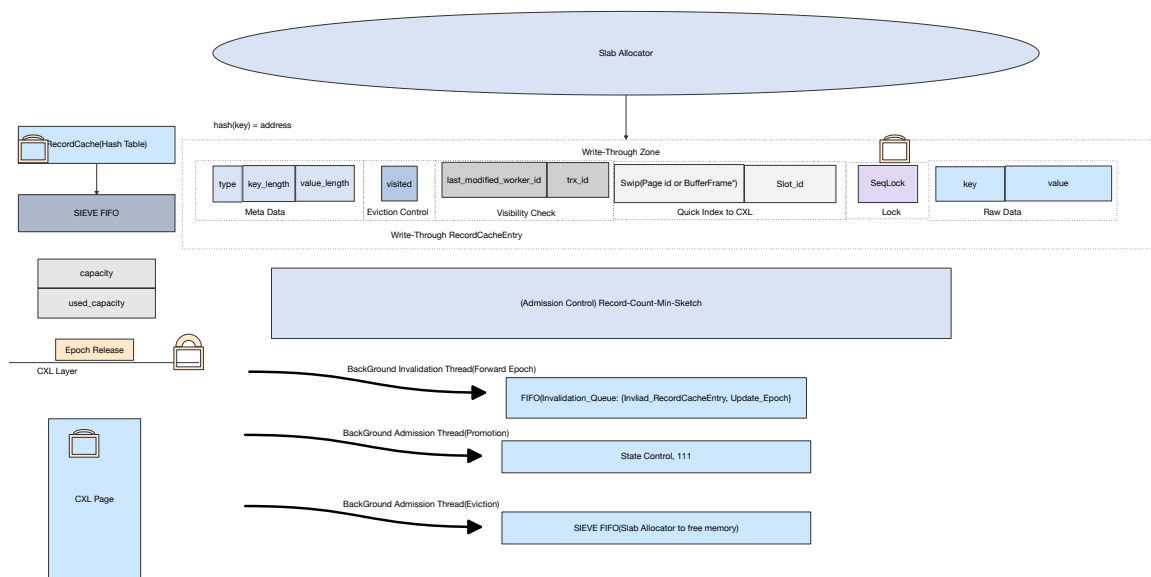
当 Swip 字段表示的页面在内存中时，记录当前 Record 在 CXL BufferPool Page 上的偏移，

用法是 (BTreeNode*)reinterpret_cast<BufferFrame.page.data>.slot[slot_id].offset

Lock:

SeqLock(std::atomic<uint64_t> 8 Byte): 乐观版本号，读操作会检测两次版本号，写操作执行原子增加指令。

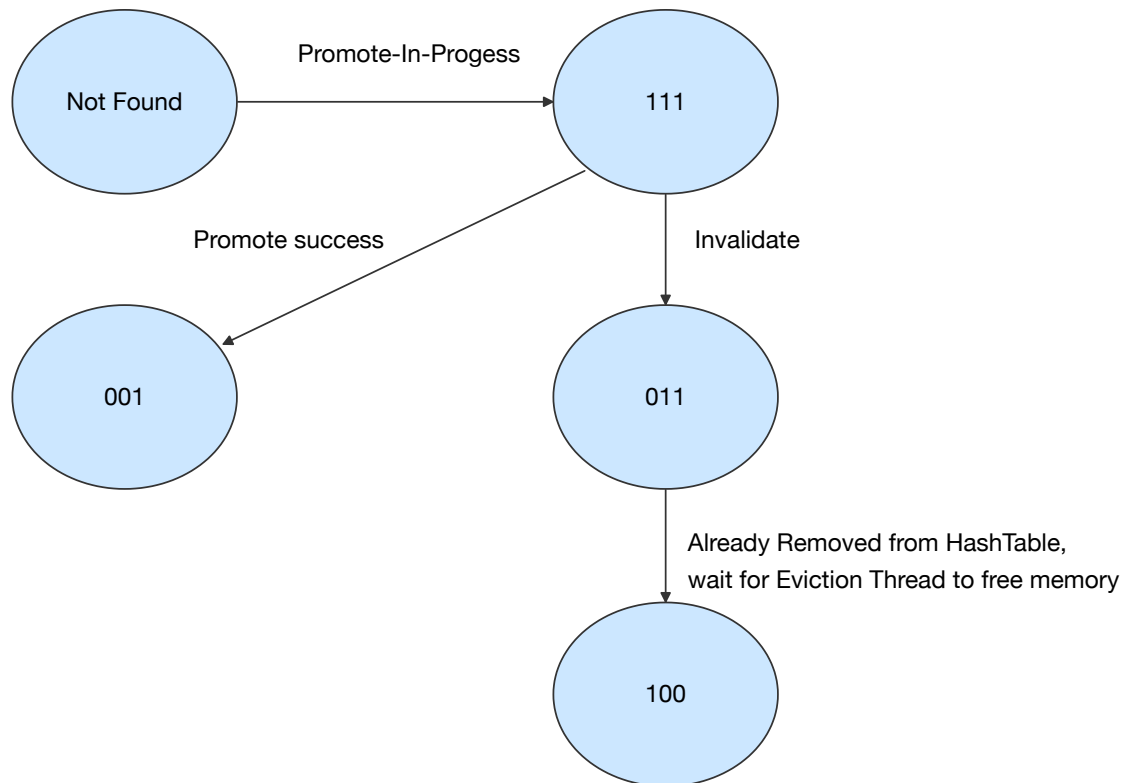
2. RecordCache



目前系统中考虑 4 种并发控制的锁或同步原语交互:

1. 基于 Epoch 机制的线程局部变量控制，用于内存回收
2. RecordCache 作为 HashTable 的 Bucket 锁，控制 Promote 和 Eviction 时，RecordCacheEntry 的插入和删除
3. RecordCacheEntry 字段的 SeqLock, 乐观版本号控制 Lookup 和 Update 的并发逻辑。
4. CXL BufferPool 上的 Page Latch(通常为 Hybrid Latch, 乐观版本号和 std::mutex 悲观锁混合，有回退和锁升级机制)。

按照轻重程度排序: Epoch < SeqLock < Bucket Lock < CXL Page Latch



3. Lookup/Update/Insert/Scan/Promote/Eviction workflow

当前开关开启后，Write-Through 模式对应的 RecordCache Type 字段初始状态为 001。

3.1 Lookup flow

1. Lookup 读取 global_epoch 到线程局部变量，记录 {active, global_lookup_epoch}
2. 计算 Hash(key), 定位到 Bucket, 遍历链表找到 Entry
 - a. Not Found
3. (2 -> Found):
 - a. 对 RecordCacheEntry 读一次 seqlock, 记录下 start_version, 如果 start_version 是奇数，立刻重试。
 - b. 确认 start_version 为偶数时，开始读取 Type 字段信息:
 - i. Type = 001, 当 visited 为 0 时标记为 1，将结果拷贝到变量。
 - ii. Type = 011, 正在执行 Update, 重试自旋等待。跳转回 3.a
 - iii. Type = 111, 正在执行 Promote, 按照 Cache Miss 的情况回退到 CXL 读路径。
(此时标记 Epoch 为 false, 不阻塞回收)

- c. (对于 Type = 001 和 Type = 010 情况),再次检查 end_version, 数据不一致则重试。
- 4. (2 -> Not Found) 回退到 CXL 读路径。(此时标记 Epoch 为 false, 不阻塞回收), 将访问记录到 Count-Min-Sketch 中, 访问次数增加, 等待触发 Promote 线程。Promote 线程可以获得 Lookup 线程传入的 {Key, Swip, Slot_id} 等信息作为参考。
- 5. Lookup 返回结果, 将线程局部变量改为 {false, global_lookup_epoch}, 可以安全地进行内存释放。

3.2 Update flow

1. Update 线程读取 global_epoch 到线程局部变量, 记录 {active, global_update_epoch}
2. 根据 Hash(Key) 判断是否存在对应的记录
 - a. 如果不存在, 直接进入 CXL BufferPool 执行对应的更新流程
 - b. 存在同时 Type = 001, 读取 Swip 和 Slot_id 到局部变量。(2 只快速查询, 不获取任何锁)
3. 更新 CXL, 先根据局部变量的 Swip 和 Slot_id 做一次匹配。如果验证失败, 则走 B+Tree 索引找到 Page, 获取 CXL Buffer Pool 的独占 X 锁。执行 WAL 写入和页面修改。(3 保证持久化和 CXL 更新完成)
4. 再次查询 RecordCache
 - a. Type = 001, 执行 seqLock.add(1), 版本号变成奇数。
将新的值拷贝到 RecordCacheEntry, 同时更新 Swip 和 Slot_id 的值。
再次确认 Type = 001 后, 执行 seqLock.add(1), 版本号变成偶数。
 - b. Type = 011, 说明已经有其他 Update 线程在执行, 退出。
 - c. Type = 111, 将 Type 标记为 100, 获取 HashTable 的 Bucket 锁之后, 执行 HashTable.remove(key, entry_pointer), 需要由 Promote 线程回收内存资源。
 - d. Not Found, 不处理任何过程
5. 释放 CXL BufferPool Page X 锁。
6. 将 Epoch 标记为 false。

3.3 Insert flow/Scan flow

这里仍然不涉及, Insert 导致的 SMO 会在 Update 检查过程中顺便修改掉过期的 Swip 和 Slot_id。

3.4 Promote flow

1. Promote 接收到信号, 线程根据对应的 {Swip, Slot_id} 执行 Promote 的逻辑判断。
2. Promote 首先从 Slab Allocator 中申请内存, 先写入 Type 字段 111, 获取 HashTable 的 bucket 锁之后, 将记录插入 HashTable 中。

3. Promote 从 CXL BufferPool 中读到完整的记录(中间有可能 Lookup 传入的 Swip 和 Slot_id 已经过时, 根据 Key 重新查找得到), 在 Promote 线程的本地维护 RecordCacheEntry 的缓存, 对应字段已经填充完毕。准备再次进入
 - i. 存在记录, Type = 011, 说明 Update 正在修改, 直接返回。
 - ii. 存在记录, Type = 111, 获取 Seqlock, 执行 atomic_add, 将内容从线程本地存储拷贝到 RecordCache 中, 拷贝时将 Type 修改为 001
 - iii. 存在记录, Type = 001, 说明已经被修改完成, 直接退出。
 - iv. 存在记录, Type = 100, 说明上一轮的 Promote 已经失效, 此时释放对应的 RecordCacheEntry 资源。
4. 释放 HashTable 的 bucket 锁, 释放 Promote 线程本地的 RecordCacheEntry 资源。

3.5 Eviction flow

1. Eviction 线程读取 global_epoch, 写入线程局部变量, 记为 {active, global_eviction_epoch}
2. Eviction 线程此时变得简单, 因为不需要再处理释放因为失效导致的内存资源, 只需要处理长期未访问的内存资源。
3. Eviction 线程从 FIFO 队列中获取 entry_pointer,

Eviction 在确认 Type 字段为 001 的情况下, 轮询所有的 Lookup 和 Update 线程的局部变量, 判断当前 Pop 出的 RecordCacheEntry 对应的 global_eviction_epoch, 没有小于当前 global_eviction_epoch 的线程状态为 active。

4. 如果判断成功, 则获取 HashTable 的 bucket 锁, 执行 HashTable.remove(key, entry_pointer), 然后调用 Slab Allocator 释放对应的物理内存资源。

3.6 Forward_epoch flow

1. Forward_epoch 只负责周期性地推进 global_epoch

Write-Back(Optional)

写回模型的目的是吸收 CXL 的写流量, 操作率先在 DRAM RecordCache 中执行完成。数据修改落盘, 之后后台线程再把 dirty record 合并到 CXL BufferPool 中。目的是事务提交过程中只涉及写 Redo log 和写 DRAM, 不需要等待 CXL 的写操作完成。

执行逻辑:

写 DRAM RecordCache -> 生成 Log(纯内存操作, Worker 线程把日志内容写入到线程局部的 Log Buffer 里, ns 级别, 内存中的 Log Buffer 中追加一条 Redo Log)

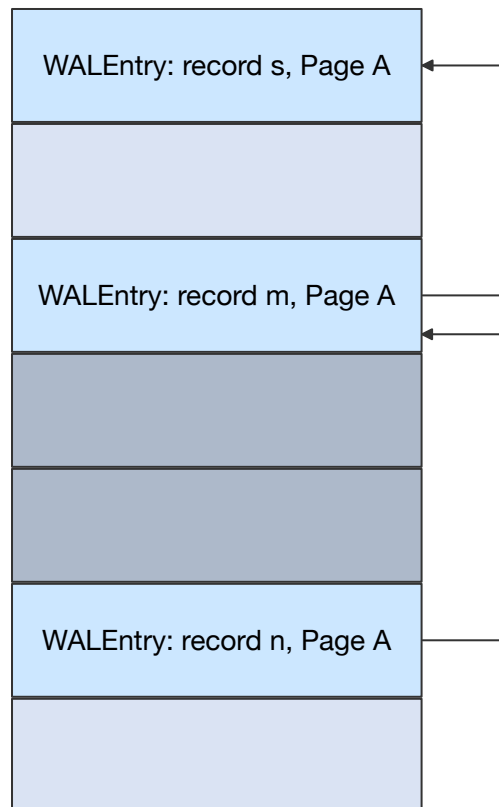
-> 写 Redo log(多个事务 Log 的Piggyback, 一次 I/O) -> 提交。

总体写事务的延迟 = 写 DRAM 的操作延迟 + 批量摊销的 Group commit 延迟。

Insight(Record-level-WAL)

关于 Write-Back, 这里可以区分出 Optimistic 的情况和 Pessimistic 的情况,

绕不开的是对 WAL 的改造, 原先的 Redo Log 是 Append-Only 的, 相当于多个 Page 的修改信息会顺序写入磁盘 IO, 本身同一个 Page 的 Insert/Update/SMO 是分散的。[16] 可以在 WALEntry 的记录上内嵌指针, 做 Single-Page-Recovery。现在以 Record 的粒度, 但是也可以内嵌指针, 保证可以找到对同一个 Page 页面的全部修改。



参考文献

- [1] Bf-Tree: A Modern Read-Write-Optimized Concurrent Larger-Than-Memory Range Index(VLDB 2024) - 16
- [2] Memtis: Efficient Memory Tiering with Dynamic Page Classification and Page Size Determination (SOSP 2023) - 98
- [3] An improved data stream summary: the count-min sketch and its Applications(DIMACS 2005) - 2858
- [4] HeavyKeeper: An Accurate Algorithm for Finding Top-k Elephant Flows(ATC 2018) - 228
- [5] The Bw-Tree: A B-tree for New Hardware Platforms(ICDE 2013) - 493

- [6] Anti-Caching: A New Approach to Database Management System Architecture(VLDB 2014) - 201
- [7] REMIX: Efficient Range Query for LSM-trees(FAST 2021) - 85
- [8] Rosetta: A Robust Space-Time Optimized Range Filters for Key-Value Stores(SIGMOD 2020) - 123
- [9] LeanStore: In-Memory Data Management beyond Main Memory(ICDE 2018) - 176
- [10] Redis: <https://github.com/redis/redis/blob/unstable/src/evict.c>
- [11] Robin-hood Hashing(unordered_dense):
https://github.com/martinus/unordered_dense
- [12] Speedy Transactions in Multicore In-Memory Databases(SOSP 2013) - 633
- [13] SIEVE is Simpler than LRU: an Efficient Turn-Key Eviction Algorithm for Web Caches (NSDI 2024) - 50
- [14] Memcached: <https://github.com/memcached/memcached/blob/master/slabs.c?spm=5176.28103460.0.0.96a07551N20QhT&file=slabs.c>
- [15] LeanStore: In-Memory Data Management beyond Main Memory(ICDE 2018) - 179
- [16] Instant recovery with write-ahead logging - 28